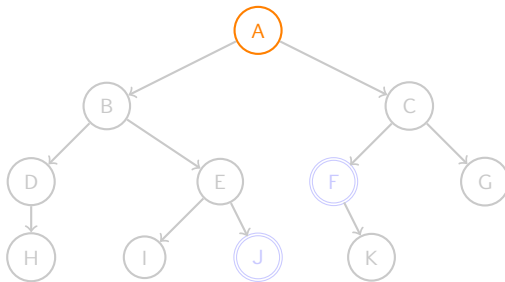


Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      | n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
-	-	-	-	{{A}}



Algorithme Largeur(Liste,Sol)

Ch ← Premier(Liste); fin ← Faux;

while Ch ≠ {} et non(fin) do

Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);

n₁ ← successeur(n);

while non(fin) et n₁ est valide do

if n₁ est solution then

| Sol ← Ch ∪_f {n₁}; fin ← Vrai;

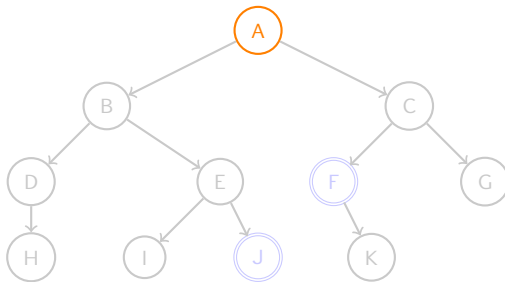
else Liste ← Liste ∪_f {(Ch ∪_f {n₁})};

n₁ ← successeur(n);

Ch ← Premier(Liste);

return fin;

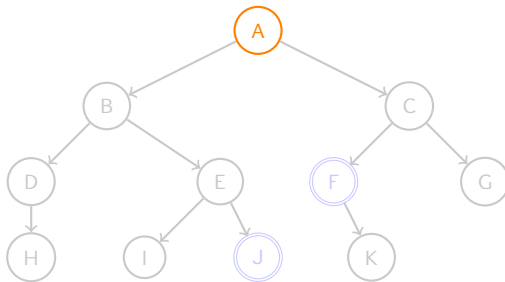
Ch	n	n ₁	fin	Liste
{A}	-	-	Faux	{{A}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

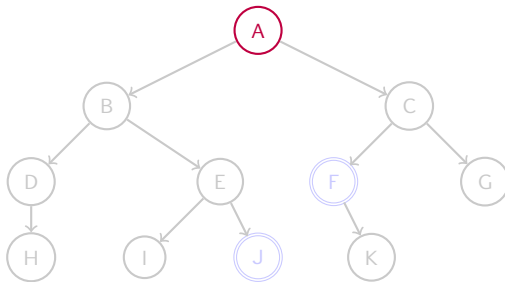
Ch	n	n_1	fin	Liste
{A}	-	-	Faux	{{A}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
    n1 ← successeur(n);  
    while non(fin) et n1 est valide do  
        if n1 est solution then  
            Sol ← Ch ∪f {n1}; fin ← Vrai ;  
        else Liste ← Liste ∪f {(Ch ∪f {n1})};  
        n1 ← successeur(n) ;  
    Ch ← Premier(Liste);  
return fin ;
```

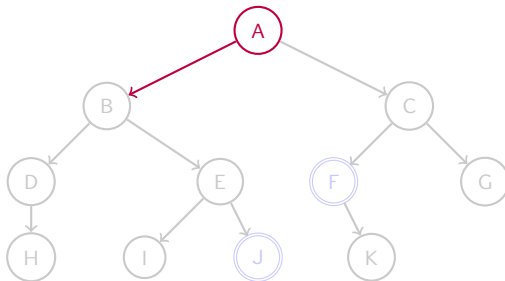
Ch	n	n_1	fin	Liste
{A}	A	-	Faux	{}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
   $n_1 \leftarrow \text{successeur}(n)$ ;  
  while non(fin) et  $n_1$  est valide do  
    if  $n_1$  est solution then  
      Sol ← Ch  $\cup_f$  { $n_1$ }; fin ← Vrai ;  
    else Liste ← Liste  $\cup_f$  {(Ch  $\cup_f$  { $n_1$ })};  
     $n_1 \leftarrow \text{successeur}(n)$  ;  
  Ch ← Premier(Liste);  
return fin ;
```

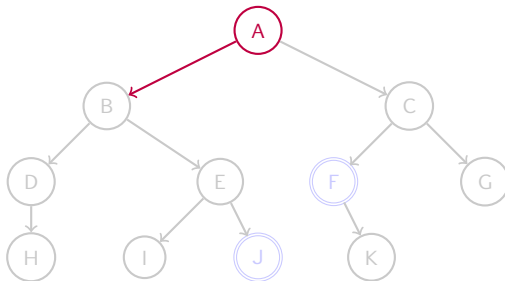
Ch	n	n_1	fin	Liste
{A}	A	B	Faux	{}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

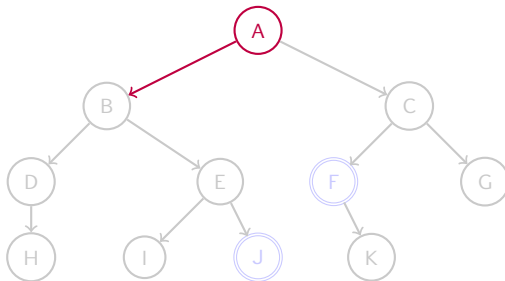
Ch	n	n_1	fin	Liste
{A}	A	B	Faux	{}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

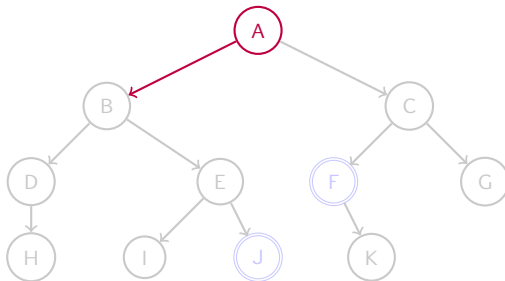
Ch	n	n_1	fin	Liste
{A}	A	B	Faux	{}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

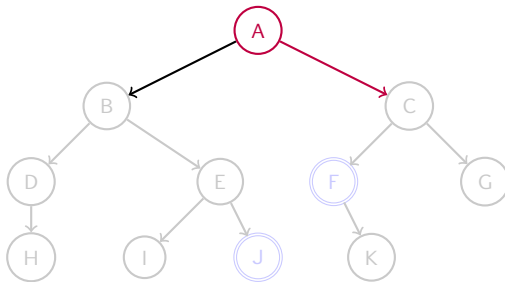
Ch	n	n_1	fin	Liste
{A}	A	B	Faux	{{AB}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

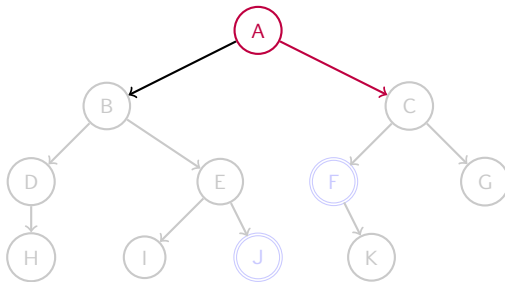
Ch	n	n ₁	fin	Liste
{A}	A	C	Faux	{{AB}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

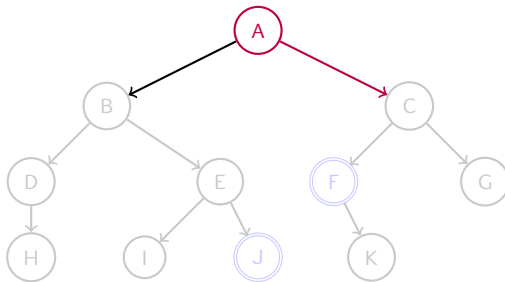
Ch	n	n_1	fin	Liste
{A}	A	C	Faux	{{AB}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

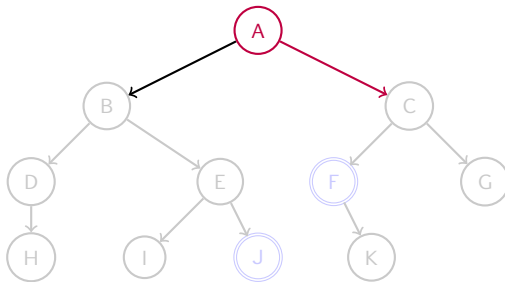
Ch	n	n_1	fin	Liste
{A}	A	C	Faux	{{AB}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪ {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪ {(Ch ∪ {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

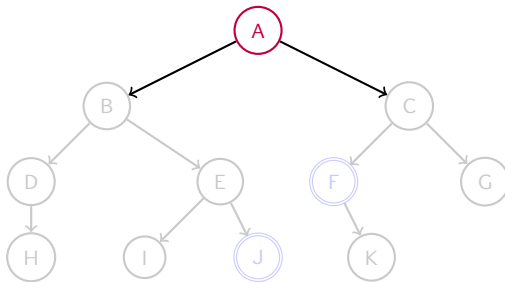
Ch	n	n_1	fin	Liste
{A}	A	C	Faux	{{AB},{AC}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

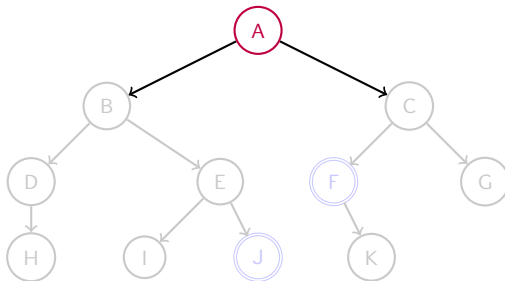
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

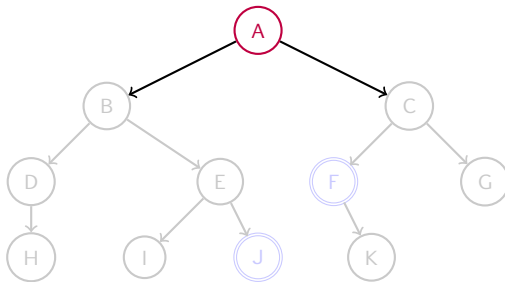
Ch	n	n_1	fin	Liste
{A}	A		Faux	{{AB},{AC}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

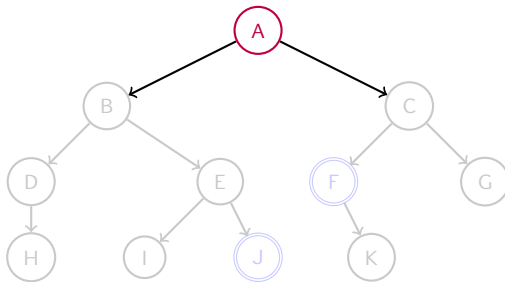
Ch	n	n_1	fin	Liste
{A} {AB}	A	$\perp$	Faux	{{AB},{AC}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪ {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪ {(Ch ∪ {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

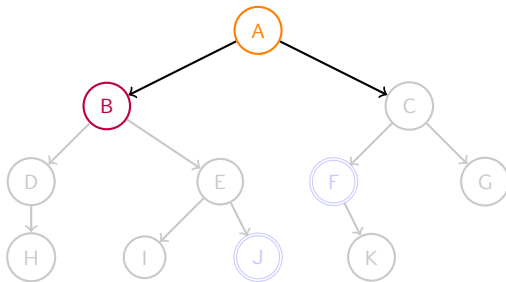
Ch	n	n_1	fin	Liste
$\{A\}$ $\{AB\}$	A	$\perp$	Faux	$\{\{AB\},\{AC\}\}$



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

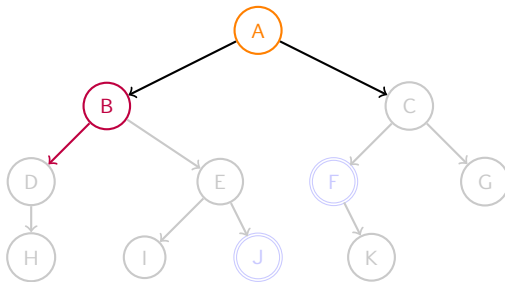
Ch	n	n_1	fin	Liste
{A} {AB}	A B	$\perp$	Faux	{{AB},{AC}} {AC}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
   $n_1 \leftarrow \text{successeur}(n)$ ;  
  while non(fin) et  $n_1$  est valide do  
    if  $n_1$  est solution then  
      | Sol ← Ch  $\cup_f$  { $n_1$ }; fin ← Vrai ;  
    else Liste ← Liste  $\cup_f$  {(Ch  $\cup_f$  { $n_1$ })};  
     $n_1 \leftarrow \text{successeur}(n)$  ;  
  Ch ← Premier(Liste);  
return fin ;
```

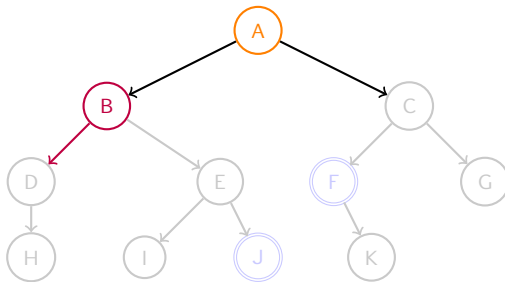
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	D		{AC}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

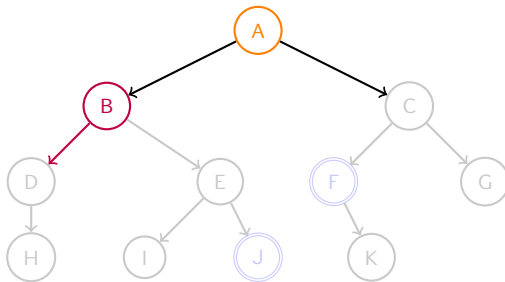
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	D		{AC}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	D		{AC}



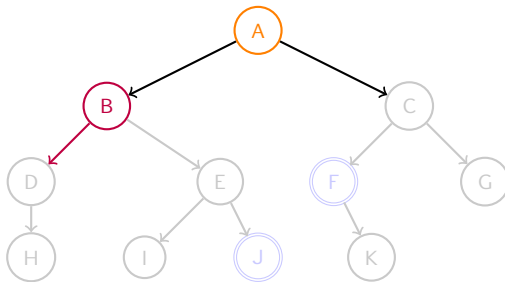
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪f {n1}; fin ← Vrai ;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

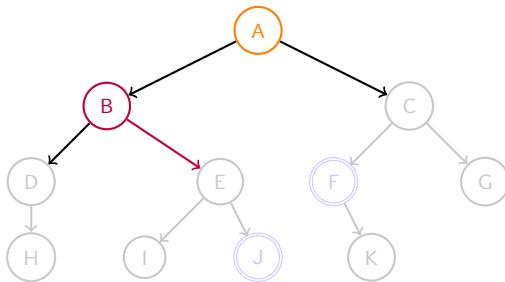
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	D		{{AC},{AC, ABD }}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

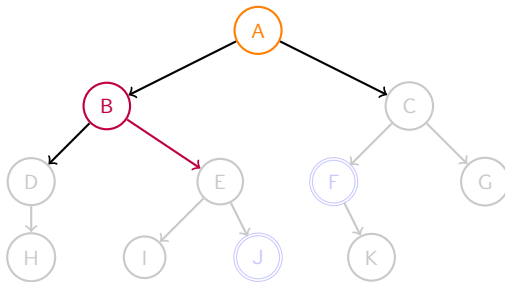
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	E		{{AC},{ABD}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

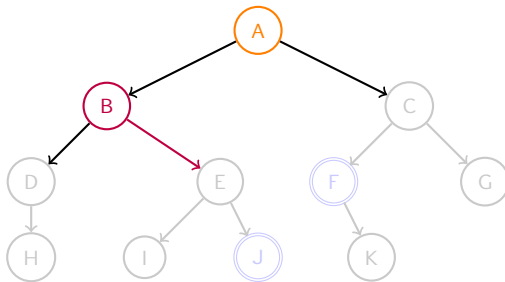
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	E		{{AC},{ABD}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	E		{{AC},{ABD}}



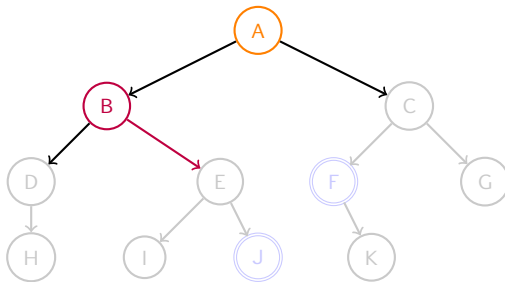
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪f {n1}; fin ← Vrai ;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

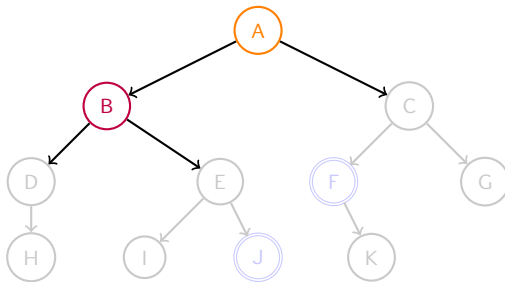
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	E		{{AC},{ABD},{ ABE }}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

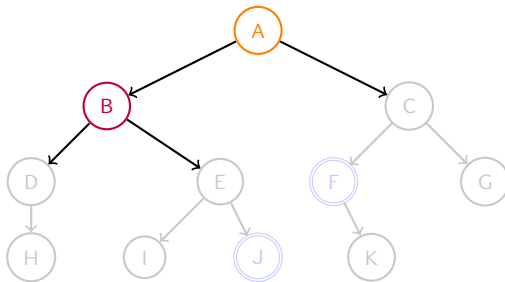
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}



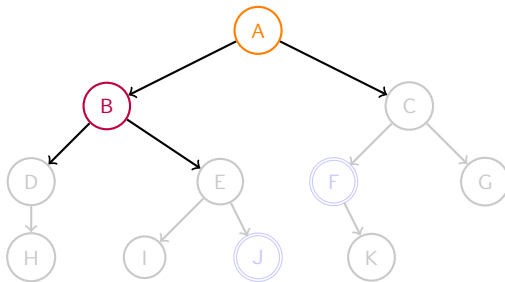
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪f {n1}; fin ← Vrai ;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}				



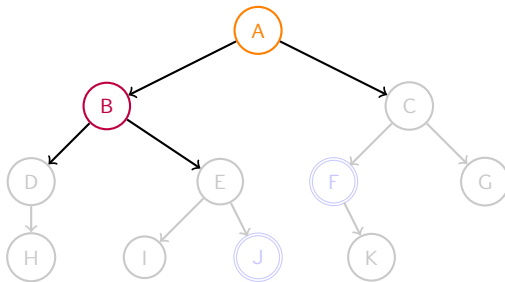
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ // et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪f {n1}; fin ← Vrai ;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}				



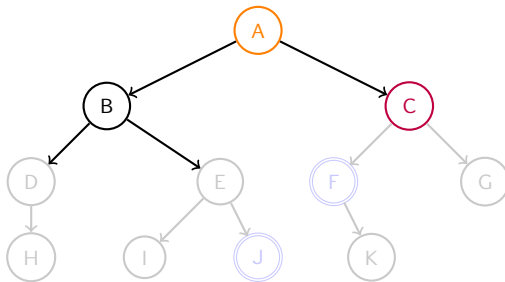
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ [] et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪f {n1}; fin ← Vrai;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C			{{ABD},{ABE}}



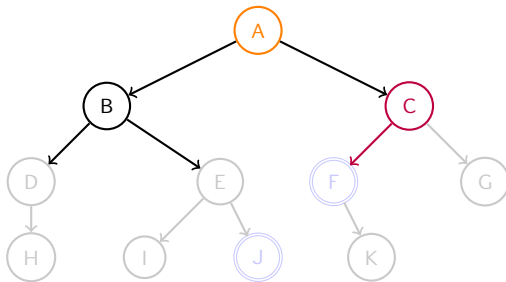
Algorithme Largeur(Liste,Sol)

```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            | Sol ← Ch ∪f {n1}; fin ← Vrai ;
        else Liste ← Liste ∪f {(Ch ∪f {n1})};
        n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

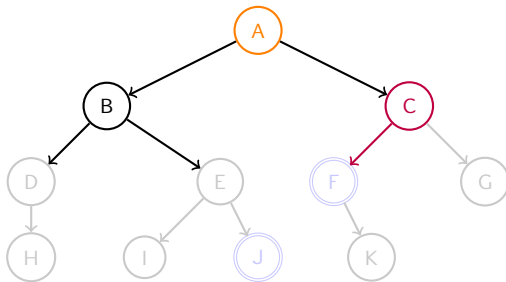
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	F		{{ABD},{ABE}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

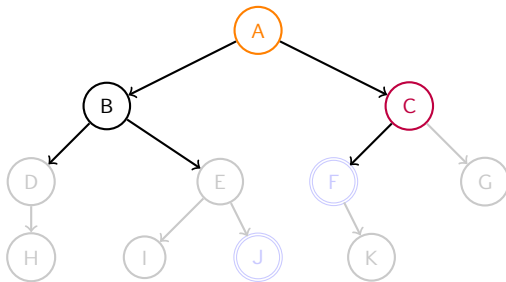
Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	F		{{ABD},{ABE}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      | Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
      | n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	F		{{ABD},{ABE}}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux;
```

```
while  $Ch \neq \{\}$  et  $non(fin)$  do
```

```
Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
```

$$n_1 \leftarrow \text{successeur}(n);$$

```
while non(fin) et  $n_1$  est valide do
```

if n_1 est solution then

Sol $\leftarrow$ Ch $\cup_f \{n_1\}$; fin $\leftarrow$ Vrai ;

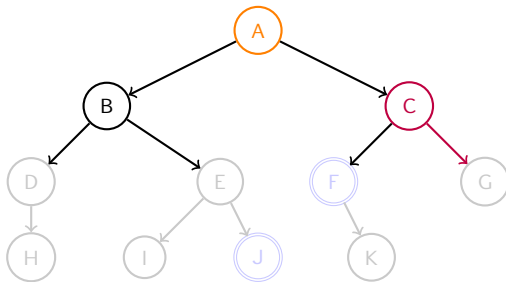
else Liste $\leftarrow$ Liste $\cup_f \{(\text{Ch} \cup_f \{n_1\})\}$;

$$n_1 \leftarrow \text{successeur}(n) ;$$

```
Ch ← Premier(Liste) ;
```

```
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	F	Vrai	{{ABD},{ABE}}

$$\text{Sol} = \{A, C, F\}$$


Algorithme Largeur(Liste,Sol)

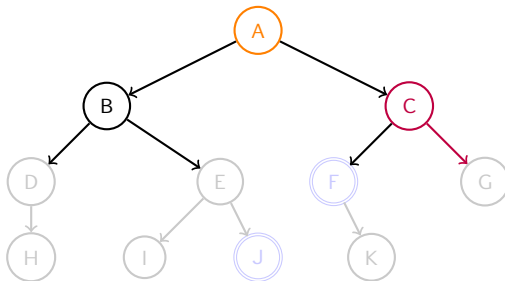
```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪ {n1}; fin ← Vrai;
        else Liste ← Liste ∪ {(Ch ∪ {n1})};
            n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	⊥	Faux	{{AB},{AC}}
{AB}	B	⊥		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

Sol = {A,C,F}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux;
```

```
while  $Ch \neq \{\}$  et  $non(fin)$  do
```

```
Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
```

$$n_1 \leftarrow \text{successeur}(n);$$

```
while non(fin) et n1 est valide do
```

if n_1 est solution then

$$\text{Sol} \leftarrow \text{Ch} \cup_f \{n_1\}; \text{fin} \leftarrow \text{Vrai};$$

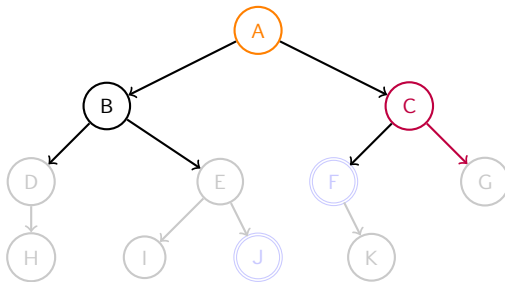
else Liste $\leftarrow$ Liste $\cup_f \{(\text{Ch} \cup_f \{n_1\})\}$;

$$n_1 \leftarrow \text{successeur}(n) ;$$

```
Ch ← Premier(Liste) ;
```

```
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

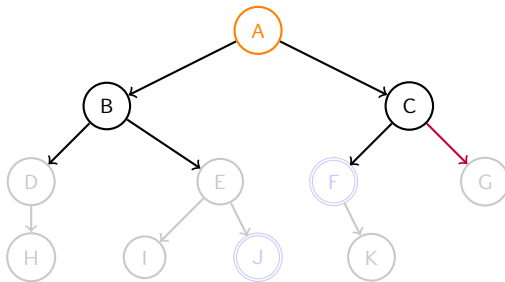
$$\text{Sol} = \{A, C, F\}$$


Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪ {n1}; fin ← Vrai;  
    else Liste ← Liste ∪ {(Ch ∪ {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

Sol = {A,C,F}



Algorithme Largeur(Liste,Sol)

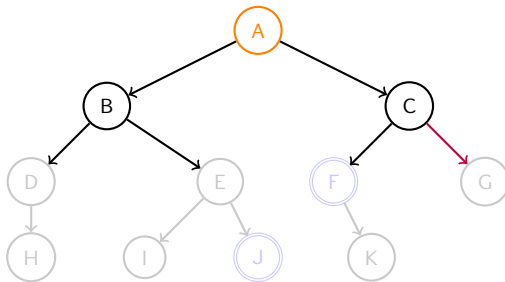
```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ // et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪ {n1}; fin ← Vrai;
        else Liste ← Liste ∪ {(Ch ∪ {n1})};
            n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

Sol = {A,C,F}



Algorithme Largeur(Liste,Sol)

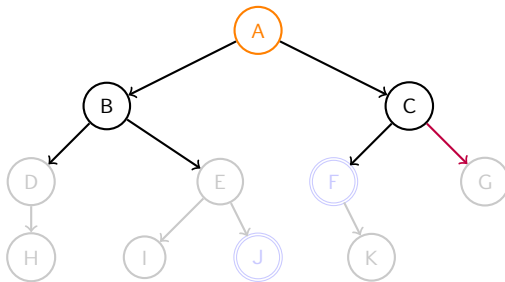
```

Ch ← Premier(Liste); fin ← Faux ;
while Ch ≠ {} et non(fin) do
    Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
    n1 ← successeur(n);
    while non(fin) et n1 est valide do
        if n1 est solution then
            Sol ← Ch ∪ {n1}; fin ← Vrai;
        else Liste ← Liste ∪ {(Ch ∪ {n1})};
            n1 ← successeur(n);
    Ch ← Premier(Liste);
return fin ;

```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

Sol = {A,C,F}



Algorithme Largeur(Liste,Sol)

```
Ch ← Premier(Liste); fin ← Faux ;  
while Ch ≠ {} et non(fin) do  
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);  
  n1 ← successeur(n);  
  while non(fin) et n1 est valide do  
    if n1 est solution then  
      Sol ← Ch ∪f {n1}; fin ← Vrai ;  
    else Liste ← Liste ∪f {(Ch ∪f {n1})};  
    n1 ← successeur(n);  
  Ch ← Premier(Liste);  
return fin ;
```

Ch	n	n_1	fin	Liste
{A}	A	$\perp$	Faux	{{AB},{AC}}
{AB}	B	$\perp$		{{AC},{ABD},{ABE}}
{AC}	C	G	Vrai	{{ABD},{ABE}}

La solution est {A,C,F}

